

Assignment 2 Perform Your Own Reconnaissance

Contents

Executive Briefing	2
Technical Briefing	2
Task 1 Scanning for Hosts	3
Fping:	3
Arp Scan:	3
NMAP	4
Task 2 OS Fingerprinting	5
NMAP active scan	5
Tcpdump: A passive OS fingerprinting method	5
ICMP Based NMAP Scan	7
Task 3 Open Port Identification	8
TCP SYN Scan (Using Nmap)	8
UDP Scan (Using Nmap)	8
Application Layer Banner Grabbing (Using Netcat)	9
Task 4 Identifying Services	10
Manual Analysis with WireShark	10
Non-Manual Analysis with Nmap	14

Executive Briefing

For this assignment the purpose was to perform various forms of reconnaissance on a network. For the purposes of this assignment, I created a network using my phone and then connected two virtual machines, my KALI Linux and a Linux mint VM. This kind of reconnaissance provides information about a network that could potentially include vulnerabilities that an attacker could use to compromise devices or the entire network. The reconnaissance starts with finding devices on the network, second identifying what types of OS the devices are running, then scanning those devices for open ports, and finally identifying services running on those hosts. In each section below I go into detail as to the information obtained, how it might be used for attackers, what defenders should look for, and finally a usefulness rank for each method.

Technical Briefing

To perform this reconnaissance, I created a separate network using my mobile device then connected two virtual machines through my machine to the network. I did this to ensure that I would have two devices always online and to also limit the number of devices I would have to handle. When I initially tried this on a home network, I found that devices would seem to come and go online or not be detectable for some reason. Also, I found that there was a staggering number of devices on the home network that I initially tried, 20+, but the number seemed to vary. The main idea behind task 1 involves using tools to scan or ping all valid addresses on the network, usually 192.168.1.1 through 192.168.1.254. .255 is not scanned because that is the broadcast address. In my network for reference 192.168.1.142 is the Linux mint virtual machine and 192.168.1.14 is the kali Linux virtual machine. Note part way through the assignment I took a break and when I returned the IP addresses had changed to .169 for the Linux mint and .101 for the kali VM. For task 2 I used NMAP to do an active scan, TCP dump to do a passive inspection of packets and ICMP fingerprinting using NMAP. For task 3 I first used a TCP SYN Scan (Using Nmap) then a UDP Scan (Using Nmap) and last Application Layer Banner Grabbing (Using Netcat). For task 4 I will be identifying services running on the target host (Linux mint box), to do this I will be using Nmap to scan for services and Wireshark to manually inspect packets for information on services. While using Nmap I was unable to find any services, for whatever reason my network configuration seems to be blocking automatic packet tools, however using wire shark I am able to see network traffic to and from the target host. In the below examples I was able to find examples of TLS, DNS, ICMP, SSH, and HTTP protocol packets and identified their usage and version. All the protocols that I found seemed to be of the latest version, but an

attacker could potentially use this information to find an exploit if any service was out of date or vulnerable. In this case because I was doing manual packet inspection it would be extremely difficult for a network defender to be aware an attacker was doing reconnaissance on the network. For each task, please see the section below for a detailed explanation of the information obtained, how it would be used by attackers, what defenders should look for, and a usefulness rank.

Task 1 Scanning for Hosts

Fping:

The first technique I will be using is fping, specifically the command

fping -a -g 192.168.1.1 192.168.1.254 to scan the entire range of addresses using a ping.

```
(root@kali)-[/home/kali]
# fping -a -g 192.168.1.1 192.168.1.254
192.168.1.14
192.168.1.142
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.2
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.2
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.2
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.2
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.1
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.1
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.1
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.1
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.4
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.4
ICMP Host Unreachable from 192.168.1.14 for ICMP Echo sent to 192.168.1.4
```

Explanation: List of IP addresses that responded to the ping, indicating active hosts.

Showing two devices, at 192.168.1.14 and 192.168.1.142.

Information Obtained: Identifies IP address of active devices within the target network for further scanning or attacking.

Indicators for defenders: Logs showing ping requests to many sequential IPs.

Usefulness Rank: 3 Because fping only provides the IP and no further information.

Arp Scan:

The second technique I will be using is an arp scan using arp-scan -l

```
(root@kali)-[/home/kali]
# arp-scan -l
Interface: eth0, type: EN10MB, MAC: 08:00:27:6e:13:6e, IPv4: 192.168.1.14
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.142    08:00:27:13:be:a7    (Unknown)

1 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.868 seconds (137.04 hosts/sec). 1 responded
```

Explanation: A table of IP addresses and their corresponding MAC addresses, along with the vendor/manufacturer information of each device. In this case it found the IP address and MAC address of the devices but was unable to open the vendor file so it couldn't get any more information.

Information Obtained: Attempts to gather the IP address, MAC address, and the device type

Indicators for defenders: Unusual ARP traffic from a single source hitting all addresses in a subnet

Usefulness Rank: 1 Because ARP attempts to gather more than the IP, it tried to get the MAC which could be used for attacks, including vendor information if it can access it.

NMAP

the last technique will be using a NMAP scan using `nmap -sn 192.168.1.0/24`

```
(root@kali)-[/home/kali]
# nmap -sn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-18 23:52 EST
Nmap scan report for 192.168.1.142
Host is up (0.00062s latency).
MAC Address: 08:00:27:13:BE:A7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.1.14
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 6.52 seconds
```

Explanation: Provides a list of IP addresses and optionally MAC addresses. It did find the two VMs at 192.168.1.14 and 192.168.1.142

Information Obtained: Attempts to collect the IP address and the MAC address. The MAC could be used for a man in the middle attack.

Indicators for defenders: Sudden spikes in network traffic targeting many or all devices on a network.

Usefulness Rank: 2 Because Nmap only provides the IP and MAC.

Task 2 OS Fingerprinting

NMAP active scan

using nmap -O 192.168.1.1

```
(root@kali)-[/home/kali]
# nmap -Pn -O 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-19 00:10 EST
Nmap done: 1 IP address (0 hosts up) scanned in 1.55 seconds

(root@kali)-[/home/kali]
# nmap -Pn -O 192.168.1.142
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-19 00:10 EST
Nmap scan report for 192.168.1.142
Host is up (0.00075s latency).
All 1000 scanned ports on 192.168.1.142 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:13:BE:A7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.65 seconds
```

Explanation: NMAP did not seem to work here as intended. At first NMAP couldn't even find the other host for some reason. I tried running the NMAP scan from my task 1 and it did find it again but when -O is added it doesn't seem to find it. When I targeted the address specifically it was able to find the host but it was unable to identify the OS because it had too many matches. The -Pn flag tag was added at the recommendation of the prompt but did not seem to make a difference.

Information Obtained: In this case my scan failed but it potentially could have found the OS name, version, and kernel version, and open ports. This would have given attackers important information on how to customize their attack for the specific OS and or ports.

Indicators for Defenders: The most obvious would be lots showing SYN/ACK packets or other unusual TCP flags.

Usefulness Rank: 1 Most effective and reliable for active OS identification. Nmap provides detailed results, making it highly useful for attackers.

Tcpdump: A passive OS fingerprinting method

using tcpdump -i eth0 -w capture.pcap then analyze with wireshark.

151	13.488378	fe80::d221:f9ff:feb...	ff02::1	UDP	250	55547	→	10001	Len=188
152	13.701112	192.168.1.237	192.168.1.255	UDP	77	44662	→	15600	Len=35
153	13.854296	192.168.1.82	255.255.255.255	UDP	232	53989	→	10001	Len=190
154	13.854297	fe80::f692:bfff:fe5...	ff02::1	UDP	252	46875	→	10001	Len=190
155	13.919646	Ubiquiti_b5:87:be	Broadcast	ARP	60	Who has	192.168.1.1?	Tell	192.168.1.36
156	14.067525	Dell_8a:f7:20	Broadcast	ARP	60	Who has	192.168.1.12?	Tell	192.168.1.107
157	14.084482	ChongqingFug_60:09:...	Broadcast	ARP	60	Who has	192.168.1.232?	Tell	192.168.1.141
158	14.152778	192.168.1.169	192.168.1.1	DNS	99	Standard query	0x57c9 A contile.services.mozilla.com OPT		
159	14.152778	192.168.1.169	192.168.1.1	DNS	99	Standard query	0x1cd4 AAAA contile.services.mozilla.com OPT		
160	14.159302	192.168.1.169	192.168.1.1	DNS	101	Standard query	0x864d A incoming.telemetry.mozilla.org OPT		
161	14.159524	192.168.1.169	192.168.1.1	DNS	101	Standard query	0x61dc AAAA incoming.telemetry.mozilla.org OPT		
162	14.172421	192.168.1.1	192.168.1.169	DNS	115	Standard query response	0x57c9 A contile.services.mozilla.com		
163	14.184881	192.168.1.1	192.168.1.169	DNS	180	Standard query response	0x1cd4 AAAA contile.services.mozilla.co		
164	14.184882	192.168.1.1	192.168.1.169	DNS	176	Standard query response	0x864d A incoming.telemetry.mozilla.org		
165	14.186026	192.168.1.169	34.117.188.166	QUIC	1294	Initial, DCID=247a2af2ef3dfaf6e7d64f68a6, SCID=15b4d2, PKN: 0,			
166	14.186935	192.168.1.1	192.168.1.169	DNS	239	Standard query response	0x61dc AAAA incoming.telemetry.mozilla		
167	14.187422	192.168.1.169	192.168.1.1	DNS	116	Standard query	0xc687 AAAA telemetry-incoming.r53-2.services.m		
168	14.189051	192.168.1.1	192.168.1.169	DNS	116	Standard query response	0xc687 AAAA telemetry-incoming.r53-2.s		
169	14.189844	192.168.1.169	34.120.208.123	TCP	74	51364	→	443	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSv
170	14.206621	34.120.208.123	192.168.1.169	TCP	74	443	→	51364	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SA
171	14.207024	192.168.1.169	34.120.208.123	TCP	66	51364	→	443	[ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1228894867
172	14.211527	34.117.188.166	192.168.1.169	QUIC	1294	Handshake, DCID=15b4d2, SCID=e47a2af2ef3dfaf6			
173	14.211528	34.117.188.166	192.168.1.169	QUIC	1294	Handshake, DCID=15b4d2, SCID=e47a2af2ef3dfaf6			
174	14.211528	34.117.188.166	192.168.1.169	QUIC	942	Protected Payload (KP0), DCID=15b4d2			
175	14.213287	192.168.1.169	34.120.208.123	TLSv1.3	2537	Client Hello (SNI=incoming.telemetry.mozilla.org)			
176	14.214052	192.168.1.169	34.117.188.166	QUIC	84	Handshake, DCID=e47a2af2ef3dfaf6, SCID=15b4d2			
177	14.226024	192.168.1.169	34.117.188.166	QUIC	173	Protected Payload (KP0), DCID=e47a2af2ef3dfaf6			
178	14.226024	192.168.1.169	34.117.188.166	QUIC	112	Protected Payload (KP0), DCID=e47a2af2ef3dfaf6			
179	14.226513	192.168.1.169	34.117.188.166	QUIC	218	Protected Payload (KP0), DCID=e47a2af2ef3dfaf6			
180	14.231102	34.120.208.123	192.168.1.169	TCP	66	443	→	51364	[ACK] Seq=1 Ack=1401 Win=267520 Len=0 TSval=200959
181	14.231102	34.120.208.123	192.168.1.169	TCP	66	443	→	51364	[ACK] Seq=1 Ack=2472 Win=266496 Len=0 TSval=200959
182	14.238492	34.120.208.123	192.168.1.169	TLSv1.3	278	Server Hello, Change Cipher Spec, Application Data			
183	14.239062	192.168.1.169	34.120.208.123	TCP	66	51364	→	443	[ACK] Seq=2472 Ack=213 Win=64128 Len=0 TSval=12288
184	14.239273	192.168.1.169	34.120.208.123	TLSv1.3	130	Change Cipher Spec, Application Data			
185	14.240749	34.120.208.123	192.168.1.169	TLSv1.3	158	Application Data			
[Coloring Rule Name: UDP]									
[Coloring Rule String: udp]									
Ethernet II, Src: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7), Dst: 7a:45:58:86:82:e6 (7a:45:58:86:82:e6)									
Destination: 7a:45:58:86:82:e6 (7a:45:58:86:82:e6)									
Source: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7)									
Type: IPv4 (0x0800)									
[Stream index: 16]									
Internet Protocol Version 4, Src: 192.168.1.169, Dst: 34.117.188.166									
0100 = Version: 4									
.... 0101 = Header Length: 20 bytes (5)									
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)									
Total Length: 70									
Identification: 0x0000 (0)									
010. = Flags: 0x2, Don't fragment									
...0 0000 0000 0000 = Fragment Offset: 0									
Time to Live: 64									
Protocol: UDP (17)									
Header Checksum: 0x993a [validation disabled]									
[Header checksum status: Unverified]									
Source Address: 192.168.1.169									
Destination Address: 34.117.188.166									
[Stream index: 22]									
User Datagram Protocol, Src Port: 54173, Dst Port: 443									
Source Port: 54173									
Destination Port: 443									
Length: 50									
Checksum: 0x26f3 [unverified]									
[Checksum Status: Unverified]									
[Stream index: 55]									
[Stream Packet Number: 5]									
[Timestamps]									
UDP payload (42 bytes)									
QUIC IETF									
QUIC Connection information									
[Packet Length: 42]									
1... = Header Form: Long Header (1)									
"S" is not a valid protocol or protocol field.									

Explanation: Using wire shark I inspected packets coming from the address that I knew to be the linux mint box, 192.168.1.169. I looked up information on the internet and found that linux uses known time to live settings and specific window sizes as well. As shown in the bottom window of wire shark this is indeed some version of linux because the TTL is 64 and the window size is 502.

Information Obtained: Specific information about the OS. This information could be used to craft OS specific attacks for this active host.

Indicators for defenders: Unexpected traffic capturing tools and high traffic coming from a single device.

Usefulness Rank: 2 Stealthy and effective in environments where traffic is already flowing. Requires some skill and can be difficult.

ICMP Based NMAP Scan

using `nmap -sn --osscan-limit --traceroute 192.168.1.x`

```
(root@kali)-[/home/kali]
# nmap -sn --osscan-limit --traceroute 192.168.1.169
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-20 08:27 EST
Nmap scan report for mint.localdomain (192.168.1.169)
Host is up (0.0014s latency).
MAC Address: 08:00:27:13:BE:A7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

TRACEROUTE
HOP RTT    ADDRESS
1   1.45 ms  mint.localdomain (192.168.1.169)

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds

(root@kali)-[/home/kali]
# █
```

Explanation: ICMP fingerprinting sends specially crafted ICMP packets (e.g., echo requests, timestamp requests) and analyzes the responses for OS-specific implementations. This correctly found the linux mint box at 192.168.1.169 and did identify that it was running mint.

Information Obtained: OS details, route latency and network topology. This could be used to find vulnerabilities or craft custom attacks for this specific host.

Indicators for Defenders: A high volume of ICMP packets originating from a single source showing sequential requests.

Usefulness Rank: 3 Useful for environments with fewer restrictions on ICMP.

Task 3 Open Port Identification

TCP SYN Scan (Using Nmap)

```
(root@kali)-[~]
# nmap -sS 192.168.1.169
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-20 09:06 EST
Nmap scan report for mint.localdomain (192.168.1.169)
Host is up (0.026s latency).
All 1000 scanned ports on mint.localdomain (192.168.1.169) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:13:BE:A7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.55 seconds
```

Explanation: Here I tried attempting a “SYN” scan using NMAP, however all 1000 ports of the target host ignored the request, IE the scan did not find any open ports. However, this scan can sometimes find open ports and is considered a relatively stealthy attack.

Information Obtained: In this case no open ports were found but it easily could have found ports that could then be used to exploitation by the attacker.

Indicators for defenders: High volume of SYN packets from a specific host to other hosts.

Usefulness Rank: 2 This method is very useful because it is relatively stealthy and does scan all ports, however as seen in this example not all ports respond in this situation.

UDP Scan (Using Nmap)

```
(root@kali)-[~]
# nmap -sU 192.168.1.169
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-20 09:07 EST
Stats: 0:05:16 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 32.80% done; ETC: 09:23 (0:10:47 remaining)
Stats: 0:15:05 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 90.60% done; ETC: 09:24 (0:01:34 remaining)
Nmap scan report for mint.localdomain (192.168.1.169)
Host is up (0.0083s latency).
Not shown: 999 closed udp ports (port-unreach)
PORT      STATE      SERVICE
5353/udp  open|filtered zeroconf
MAC Address: 08:00:27:13:BE:A7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1009.19 seconds
```

Explanation: This is sending UDP packets to a targets ports to perform a scan. In this case it was able to identify a closed but usable port 5353 on the target host. However this method is considered less stealthy.

Information Obtained: This method would give the attacker information about ports open or closed, which could be used to craft a customized attack on the host.

Indicators for defenders: Unexpected UDP traffic patterns and high volumes of “port unreachable” messages.

Usefulness Rank: 1 This method while less stealthy provides information on ports even if they are closed, which gives more information about the target host.

Application Layer Banner Grabbing (Using Netcat)

```
(kali㉿kali)-[~]  
$ sudo -s  
[sudo] password for kali:  
zsh: corrupt history file /root/.zsh_history  
(root㉿kali)-[/home/kali]  
# nc -v 192.168.1.169 22  
mint.localdomain [192.168.1.169] 22 (ssh) : Connection refused  
  
(root㉿kali)-[/home/kali]  
# nc -v 192.168.1.169 80  
mint.localdomain [192.168.1.169] 80 (http) : Connection refused  
  
(root㉿kali)-[/home/kali]  
# █
```

Explanation: Here I manually connect to an open port to send simple requests or inspect responses. In this case the open ports that I tried were suggestions from the internet however the target host refused all the ports that I attempted. This makes sense unfortunately because the other methods did not find any open ports.

Information Obtained: In this case none, however, this method could potentially identify open ports and even information on the service using the port.

Indicators for defenders: Connection logs showing unusual or incomplete requests.

Usefulness Rank: 3 This method does provide information however with it being a manual method it is more difficult.

Task 4 Identifying Services

Manual Analysis with Wireshark

Below are services found using Wireshark to do manual packet inspection.

TLS

In this example I initiated a connection to Wikipedia which created this packet traffic. The first example I found was this packet using TLS to establish the handshake between the target active host and wikipedia.org. This service is using version 1.3 which is the latest version of TLS. I was able to verify the version of this service because Wireshark is able to identify it.

283	21.151540776	192.168.1.169	198.35.26.112	TLSv1.3	1967 Client Hello (SNI=upload.wikimedia.org)
284	21.152144090	198.35.26.96	192.168.1.169	TCP	2962 443 → 43878 [PSH, ACK] Seq=10523 Ack=3525 Win=42496 Len=2896
285	21.152144167	198.35.26.96	192.168.1.169	TCP	1514 443 → 43878 [ACK] Seq=13419 Ack=3525 Win=42496 Len=1448 TSval=
286	21.152501092	192.168.1.169	198.35.26.96	TCP	66 43878 → 443 [ACK] Seq=3525 Ack=13419 Win=80896 Len=0 TSval=10
287	21.153575571	198.35.26.96	192.168.1.169	TCP	1514 443 → 43878 [PSH, ACK] Seq=14867 Ack=3525 Win=42496 Len=1448
288	21.153575635	198.35.26.96	192.168.1.169	TCP	1514 443 → 43878 [ACK] Seq=16315 Ack=3525 Win=42496 Len=1448 TSval=
289	21.155243681	192.168.1.169	198.35.26.96	TCP	66 43878 → 443 [ACK] Seq=3525 Ack=17763 Win=80896 Len=0 TSval=10

▶ Frame 283: 1967 bytes on wire (15736 bits), 1967 bytes captured (15736 bits) on interface eth0, id 0

▶ Ethernet II, Src: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7), Dst: 7a:45:58:86:e6 (7a:45:58:86:e6)

▶ Internet Protocol Version 4, Src: 192.168.1.169, Dst: 198.35.26.112

▶ Transmission Control Protocol, Src Port: 60754, Dst Port: 443, Seq: 1, Ack: 1, Len: 1901

Source Port: 60754

Destination Port: 443

[Stream index: 1]

▶ [Conversation completeness: Incomplete, DATA (15)]

[TCP Segment Len: 1901]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 3263504604

[Next Sequence Number: 1902 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 62896819

1000 = Header Length: 32 bytes (8)

▶ Flags: 0x018 (PSH, ACK)

Window: 502

[Calculated window size: 64256]

[Window size scaling factor: 128]

Checksum: 0xaa78 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

▶ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

▶ [Timestamps]

▶ [SEQ/ACK analysis]

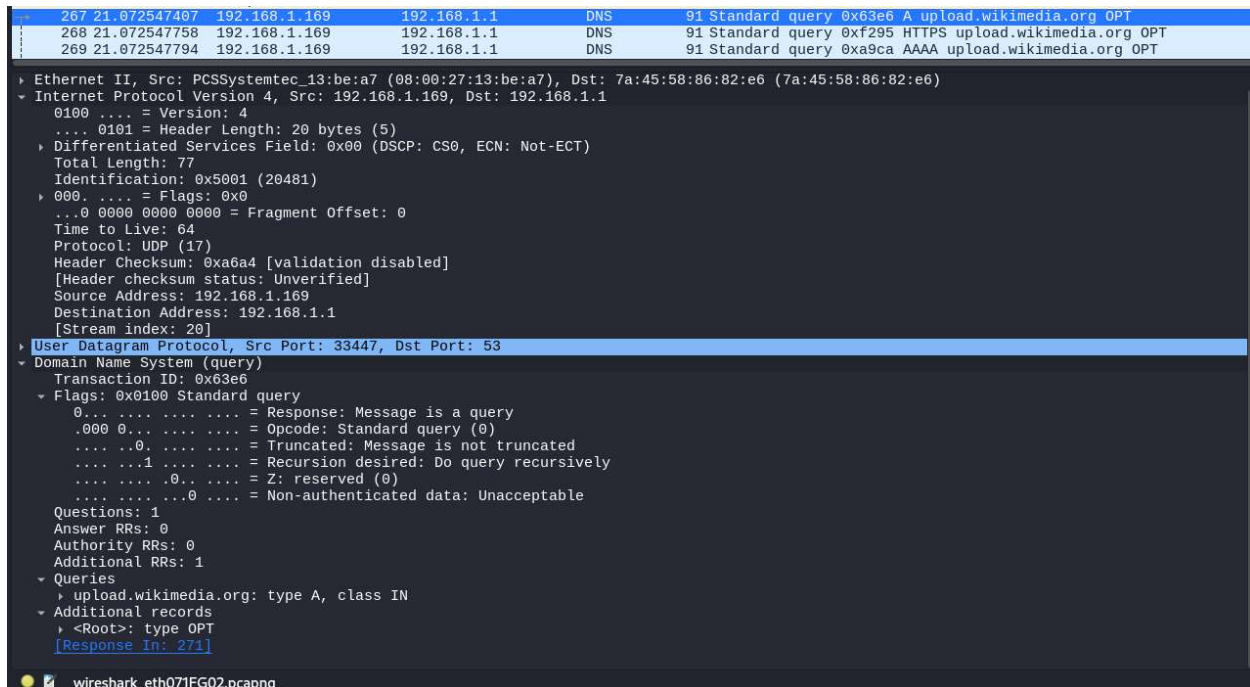
TCP payload (1901 bytes)

▶ Transport Layer Security

▶ TLSv1.3 Record Layer: Handshake Protocol: Client Hello

DNS

In this example this is again from initiating a connection to Wikipedia. This packet inspection shows the DNS look up for the Wikipedia site address.



```
267 21.072547407 192.168.1.169 192.168.1.1 DNS 91 Standard query 0x63e6 A upload.wikimedia.org OPT
268 21.072547758 192.168.1.169 192.168.1.1 DNS 91 Standard query 0xf295 HTTPS upload.wikimedia.org OPT
269 21.072547794 192.168.1.169 192.168.1.1 DNS 91 Standard query 0xa9ca AAAA upload.wikimedia.org OPT

Ethernet II, Src: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7), Dst: 7a:45:58:86:82:e6 (7a:45:58:86:82:e6)
Internet Protocol Version 4, Src: 192.168.1.169, Dst: 192.168.1.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 77
  Identification: 0x5001 (20481)
  0000 .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: UDP (17)
  Header Checksum: 0xa6a4 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.169
  Destination Address: 192.168.1.1
  [Stream index: 20]
User Datagram Protocol, Src Port: 33447, Dst Port: 53
Domain Name System (query)
  Transaction ID: 0x63e6
  Flags: 0x0100 Standard query
    0... .. = Response: Message is a query
    .000 0... .. = Opcode: Standard query (0)
    .... 0... .. = Truncated: Message is not truncated
    .... 1... .. = Recursion desired: Do query recursively
    .... 0... .. = Z: reserved (0)
    .... 0... .. = Non-authenticated data: Unacceptable
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 1
  Queries
    upload.wikimedia.org: type A, class IN
  Additional records
    <Root>: type OPT
  [Response In: 271]
```

ICMP

Here is an example of an attempt of an ICMP packet between the two virtual machines that I am running, ending in .169 and .101. For whatever reason, these packets are being dropped and the active host attempts to resend this packet on a regular frequency. I suspect this is a symptom of why I am having network issues between the two virtual machines however I do not fully understand the cause.

```

263 20.846278733 192.168.1.169 192.168.1.101 ICMP 70 Destination unreachable (Port unreachable)
.... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
  Total Length: 56
  Identification: 0x8f69 (36713)
  0000 .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x663d [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.169
  Destination Address: 192.168.1.101
  [Stream index: 0]
  Internet Control Message Protocol
    Type: 3 (Destination unreachable)
    Code: 3 (Port unreachable)
    Checksum: 0x8175 [correct]
    [Checksum Status: Good]
    Unused: 00000000
  Internet Protocol Version 4, Src: 192.168.1.101, Dst: 192.168.1.169
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 28
    Identification: 0x48a1 (18593)
    0000 .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 43
    Protocol: UDP (17)
    Header Checksum: 0xc2d1 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.1.101
    Destination Address: 192.168.1.169
    [Stream index: 0]
  User Datagram Protocol, Src Port: 55162, Dst Port: 2292

```

SSH

Here I initiated an SSH connection to the Oregon state access server. Wire shark was able to identify that the service is SSH and is using version 2 which is the current version of SSH.

```

2065 144.043284447 192.168.1.169 128.193.54.168 SSHv2 338 Client: Encrypted packet (len=272)
2066 144.067238985 128.193.54.168 192.168.1.169 TCP 66 22 → 46745 [ACK] Seq=2510 Ack=2621
2067 144.211638963 ChongqingFuo 60:09: Broadcast ARP 60 Who has 192.168.1.232? Tell 192.168
  Frame 2065: 338 bytes on wire (2704 bits), 338 bytes captured (2704
  Ethernet II, Src: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7), Dst: 7i
  Internet Protocol Version 4, Src: 192.168.1.169, Dst: 128.193.54.16i
  Transmission Control Protocol, Src Port: 46745, Dst Port: 22, Seq: :
  SSH Protocol
    SSH Version 2 (encryption:aes256-ctr mac:hmac-sha2-256 compressio
    [Direction: client-to-server]
    0000 7a 45 58 86 82 e6 08 00 27 13 be a7
    0010 01 44 eb dd 40 00 40 06 d4 1b c0 a8
    0020 36 a8 b6 99 00 16 23 62 b6 0f c4 9a
    0030 01 e7 9a 33 00 00 01 01 08 0a 26 64
    0040 ec 2c 43 41 67 75 39 42 da 5e 07 01
    0050 20 d6 47 0c 04 88 40 f2 a5 cc 37 8b
    0060 16 5a 1d 71 54 b7 f1 30 98 83 e2 29
    0070 8f 95 69 5a d8 17 2c 97 0b de dc 65
    0080 08 8d cf 31 a8 e8 f6 9a 07 59 ed e5
    0090 cc ae d4 be b3 89 0e 04 fb ce 59 64
    00a0 50 d7 24 57 b2 25 28 e9 97 39 d1 71
    00b0 74 fd 1c ae 87 49 bc 59 fc 6a 09 ef
    00c0 c2 08 19 33 be 26 09 85 01 40 bb 4f
    00d0 e7 3e 6a 7b 28 2c 81 37 81 b3 d9 e6
    00e0 2b 8b c6 ee cf fa 98 79 c2 3e 76 a9
    00f0 c7 3d f6 99 9d 3a 22 1f 61 b2 90 ad
    0100 36 52 50 8d cb db 72 0c db 0d 4f eb
    0110 06 ce ca f2 52 41 b5 42 0d a4 ef a1
    0120 16 d3 32 95 b3 96 cf e7 06 ee 37 10
    0130 5f d9 be e5 c2 b9 50 f1 16 ad 1e e3
    0140 fe 5f c0 31 2b 85 8b cd bc af 87 e6
    0150 5a 37

```

HTTP

Here is an example where I had the target host install a package and this shows the packet traffic of initiating that download. One of the first things it does is use HTTP for transferring hypertext between the server and this target host.

```

337 17.412086552 192.168.1.169 91.189.91.81 HTTP 229 GET /ubuntu/pool/universe/libl/liblinear/liblinear4_2.3.
338 17.499593454 Ubiquiti_a3:a5:c0 Broadcast ARP 60 Who has 192.168.1.1? Tell 192.168.1.91
339 17.499963742 91.189.91.81 192.168.1.169 TCP 66 80 → 37080 [ACK] Seq=1 Ack=164 Win=81920 Len=0 TSval=147
340 17.499963786 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=1 Ack=164 Win=81920 Len=2896 T
341 17.499963815 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=2897 Win=69888 Len=0 TSval=
342 17.502432460 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=2897 Ack=164 Win=81920 Len=289
343 17.502432521 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=5793 Ack=164 Win=81920 Len=289
344 17.502432547 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=8689 Ack=164 Win=81920 Len=289
345 17.502432565 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=11585 Ack=164 Win=81920 Len=28
346 17.502822145 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=5793 Win=75776 Len=0 TSval=
347 17.502822169 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=8689 Win=81536 Len=0 TSval=
348 17.502822199 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=11585 Win=84352 Len=0 TSval=
349 17.502822217 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=14481 Win=81664 Len=0 TSval=
350 17.586277674 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=14481 Ack=164 Win=81920 Len=28
351 17.586465659 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=17377 Win=89856 Len=0 TSval=
352 17.589500524 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=17377 Ack=164 Win=81920 Len=28
353 17.589735828 192.168.1.169 91.189.91.81 TCP 66 37080 → 80 [ACK] Seq=164 Ack=20273 Win=89856 Len=0 TSval=
354 17.595672199 91.189.91.81 192.168.1.169 TCP 2962 80 → 37080 [PSH, ACK] Seq=20273 Ack=164 Win=81920 Len=28

Frame 337: 229 bytes on wire (1832 bits), 229 bytes captured (1832 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_13:be:a7 (08:00:27:13:be:a7), Dst: 7a:45:58:86:82:e6 (7a:45:58:86:82:e6)
Internet Protocol Version 4, Src: 192.168.1.169, Dst: 91.189.91.81
Transmission Control Protocol, Src Port: 37080, Dst Port: 80, Seq: 1, Ack: 1, Len: 163
  Source Port: 37080
  Destination Port: 80
  [Stream index: 0]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 163]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 260845628
  [Next Sequence Number: 164 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 3037470827
  1000 ... = Header Length: 32 bytes (8)
  Flags: 0x018 (PSH, ACK)
  Window: 502
  [Calculated window size: 64256]
  [Window size scaling factor: 128]
  Checksum: 0x748f [Unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  [Timestamps]
  [SEQ/ACK analysis]
  TCP payload (163 bytes)
Hypertext Transfer Protocol
  GET /ubuntu/pool/universe/libl/liblinear/liblinear4_2.3.0%2bdfsg-5build1_amd64.deb HTTP/1.1\r\n
  Host: archive.ubuntu.com\r\n
  User-Agent: Debian APT-HTTP/1.3 (2.7.14)\r\n
  \r\n
  [Response in frame: 369]
  [Full request URI: http://archive.ubuntu.com/ubuntu/pool/universe/libl/liblinear/liblinear4_2.3.0%2bdfsg-5build1_amd64.deb]

Hypertext Transfer Protocol (http), 163 bytes
Packets: 1638 - Dropped: 0 (0.0%)
Profile: D

```

As far as I can tell all of these services are running the most up to date version of said service. In these examples an attacker could potentially identify services that might be vulnerable to exploitation if any were not using the latest version. Because these services were identified using manual packet inspection it would be extremely difficult for a network defender to notice that an attacker was inspecting the network. This method can be extremely useful because it provides a lot of information about activity between hosts on the network, however this method is also extremely tedious as even a basic network has an a lot of packet traffic. In terms of usefulness, I ranked this method as 1 because it was usable in my situation where as the none manual method did not work. I also believe that manual packet inspection can provide more information if you understand the information you are looking at, albeit that is not always easy.

Non-Manual Analysis with Nmap

For whatever reason my nonmanual packet inspection is not working. Nmap and other scanning tools seem to report no open packets or ports, which is extremely strange. I investigated this with the TA and we were unable to understand why the services did not appear. While doing this investigation we were able to confirm that network traffic was occurring using wire shark, but Nmap continues to report nothing. This most likely has something to do with the network configuration that I am using, I am running two VM(s) on the same device and attempting to scan network traffic between them, but something seems to be interfering with Nmap. In terms of usefulness, I ranked this method as 2nd only because it did not work for me, and I was unable to gather any information using automatic methods. However potentially nonmanual methods could be useful only because they are so much easier to use compared to manual methods.